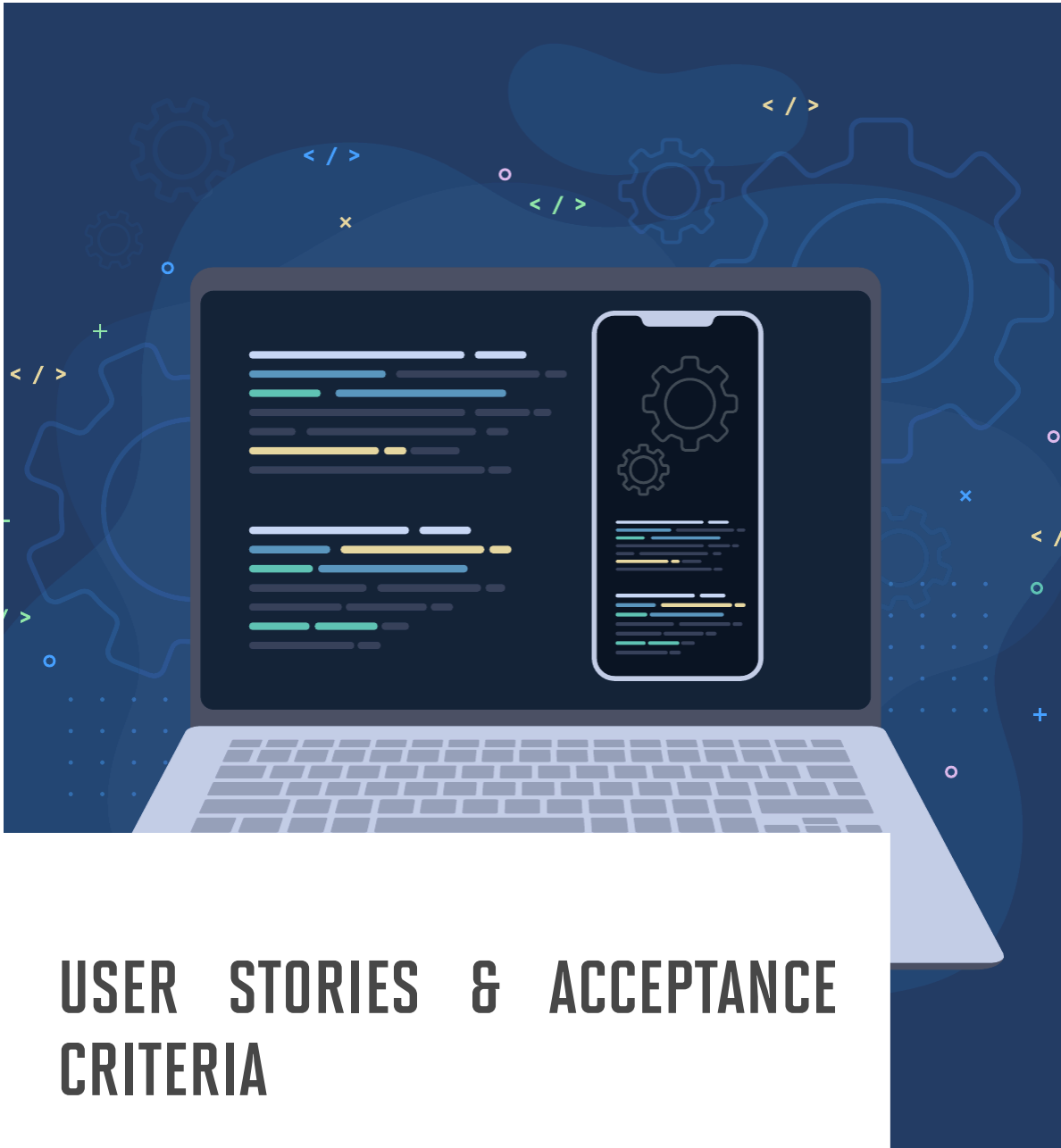




USER STORIES & ACCEPTANCE CRITERIA

Material producido por la Universidad Peruana de Ciencias Aplicadas
UPC, 2021.



CONTENIDO

Objetivo de aprendizaje	3
Introducción	3
Acceptance criteria	5
Scenarios	7
Behavior-Driven Development	9
Tips & best practices	11
Conclusiones	14
Referencias	14



Ya conocemos el formato:

"As a (type of user), **I want to** (perform some action) **so that I** (can achieve some goal/result/value)."

Para los equipos ágiles, User Stories son el primer método para identificar necesidades de los usuarios.



¿Cómo nos aseguramos de que las User Stories se han completado correctamente y conforme a las demandas de los clientes?

Acceptance criteria

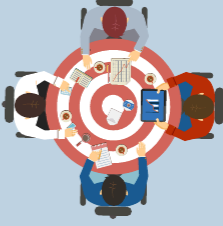
3. ACCEPTANCE CRITERIA (CRITERIOS DE ACEPTACIÓN)

- Son una lista formalizada de condiciones que permiten asegurar que todos los escenarios se han tomado en cuenta y se ha completado una User Story.
- Criterios expresados de forma concisa por escrito, ayudan al equipo a evitar ambigüedad sobre las necesidades de los clientes y evitan problemas de comunicación.
- Permiten establecer la visión de un producto, incluso ayudan en el proceso de desarrollo.

3.1 UTILIDAD

Definir alcance	• Ayudan al equipo a definir los límites de una user story. Permiten confirmar cuándo la aplicación funciona como se desea, lo que significa que la user story se ha completado.
Lograr consenso	• Tener acceptance criteria sincroniza el equipo de desarrollo con el cliente. El equipo sabe con exactitud qué condiciones se deberían cumplir, a la vez que el cliente sabe qué esperar de la app.
Servir de base para tests	• Son la piedra angular de pruebas positivas y negativas que buscan verificar si el sistema trabaja como se espera.
Permitir planificación y estimaciones precisas	• Los escenarios de acceptance criteria facilitan que las user stories se dividan en tareas que se puedan estimar y planificar.



	Who writes • Puede ser el cliente o el equipo de desarrollo. • Criterios escritos por el product owner (cliente) son revisados por un miembro del equipo de desarrollo, para asegurarse que los criterios están especificados con claridad y que no hay restricciones técnicas o inconsistencias desde la perspectiva del desarrollo. • Este flujo aplica a colaboraciones donde el product owner tiene experiencia en proyectos de desarrollo de software y tiene conocimiento sobre documentación de proyectos. • Cuando los criterios son escritos por el equipo de desarrollo, entonces un analista de requisitos, project manager o especialista de QA debería encargarse de esta tarea.
When • Deberían especificarse antes y nunca después que el desarrollo ha comenzado. De esta manera, el equipo y el product owner estarían de acuerdo sobre la mínima cantidad de entregas que satisfacen los requisitos del product owner.	
	How to write Tipos: • Rules-oriented. En forma de lista. • Scenario-oriented. En forma de escenarios que ilustran cada criterio.

4. SCENARIOS

4.1 SCENARIO-ORIENTED APPROACH

Ayuda a captar de forma rápida los requisitos, contemplar varios casos de uso y además utilizar los escenarios para pruebas de aceptación manuales y automatizadas.

- Es el approach preferido por equipos ágiles.
- Template común: Given / When / Then





Como usuario del website, deseo poder proporcionar feedback, para que mi opinión sea considerada por los dueños del website en futuras actualizaciones del mismo.

Scenario: usuario visualiza formulario “Brindar feedback”.

Dado que el usuario cuenta con el rol de usuario autenticado o invitado

Cuando el usuario abre la página “Feedback”

Entonces el sistema muestra el formulario “Brindar feedback” conteniendo los campos “Email”, “Nombre” y “Comentario” los cuales son requeridos.

Scenario: usuario envía el formulario de feedback con datos válidos.

Dado que el sistema muestra el formulario “Brindar feedback” conteniendo los campos “Email”, “Nombre” y “Comentario” los cuales son requeridos.

Y el usuario coloca una dirección de email válida en el campo “Email”

Y el usuario coloca su nombre en el campo “Nombre”

Y el usuario coloca su comentario en el campo “Comentario”

Cuando el usuario ordena “Enviar feedback”

Entonces el sistema envía mi feedback

Y el sistema muestra el mensaje “Su feedback fue enviado con éxito”

Y el sistema limpia los campos del formulario “Brindar feedback”.

5. BEHAVIOR-DRIVEN DEVELOPMENT (BDD)

- Es un proceso de desarrollo de software que coloca en primer lugar al comportamiento.
- Un behavior (comportamiento) es cómo un feature que opera dentro de un escenario bien definido de entradas, acciones y resultados.
- Los comportamientos se identifican utilizando specification by example.
- Behavior specs se convierten en los requisitos, los criterios de aceptación y las pruebas de aceptación.

5.1 BDD TEST FRAMEWORKS

- Test frameworks también pueden automatizar specs – specs declarativos y que deberían cubrir unidades de cobertura para comportamientos únicos del producto.
- Los BDD test frameworks más conocidos son Cucumber y sus derivados, que escriben specs el lenguaje Gherkin, con el estilo “Given-When-Then”.



Language	Framework	Type
C	Catch	In-line Spec
C++	Igloo	In-line Spec
C# and .NET	Concordion	In-line Spec
	LightBDD	In-line Gherkin
	NBehave x	Separated semi-Gherkin
	NSpec	In-line Spec
	SpecFlow *	Separated Gherkin
	xBehave.net	In-line Gherkin
Golang	Ginkgo	In-line Spec
Java and JVM	Cucumber-JVM *	Separated Gherkin
	JUnit	Separated semi-Gherkin
	JDave x	In-line Spec
	JGiven *	In-line Gherkin
	Scalatest	In-line Spec
	Spock	In-line Spec
JavaScript	Cucumber.js *	Separated Gherkin
	Yadda	Separated semi-Gherkin
	Jasmine	In-line Spec
	Mocha	In-line Spec
	Vows	In-line Spec
Perl	Test::BDD::Cucumber	Separated Gherkin
PHP	Behat	Separated Gherkin
	Codeception *	Separated or In-line
Python	behave *	Separated Gherkin
	freshen x	Separated Gherkin
	lettuce	Separated semi-Gherkin
	pyspecs	In-line Spec
	pytest-bdd *	Separated semi-Gherkin
	radish	Separated Gherkin-plus
Ruby	Cucumber *	Separated Gherkin
	RSpec	In-line Spec
	Spinach	Separated Gherki
Swift / Objective C	Quick	In-line Spec



6.3 BEST PRACTICES

Proper behavior

- Error típico: escribir sin un pensamiento orientado al comportamiento (procedure-driven en vez de behavior-driven).
- Evita los procedure-driven criterios, que son a menudo imperativos y trazan una ruta para el sistema que cubre múltiples comportamientos, lo cual resulta en extensión innecesaria, retrasa el análisis de fallas, incrementa costos de mantenimiento y crea confusión.

✗
Bad example! Do not copy
✗

- **Feature:** Google Searching
- **Scenario:** Google Image search shows pictures
- **Given** the user opens a web browser
- **And** the user navigates to **"https://www.google.com/"**
- **When** the user enters **"panda"** into the search bar
- **Then** links related to **"panda"** are shown on the results page
- **When** the user clicks on the **"Images"** link at the top of the results page
- **Then** images related to **"panda"** are shown on the results page

✓
The right way
✓

- **Feature:** Google Searching
- **Scenario:** Search from the search bar
- **Given** a web browser is at the Google home page
- **When** the user enters **"panda"** into the search bar
- **Then** links related to **"panda"** are shown on the results page
- **Scenario:** Image search
- **Given** Google search results for **"panda"** are shown
- **When** the user clicks on the **"Images"** link at the top of the results page
- **Then** images related to **"panda"** are shown on the results page



Phrasing steps

- Cómo escribir un paso es importante. Un paso escrito de forma deficiente es difícil que se pueda reutilizar.
- **Escribe todos los pasos en tercera persona.** Solo usa tercera persona siempre.
- **Escribe los pasos como una frase que denote acción con sujeto y predicado.** Las frases parciales generan ambigüedad e incluso se podrían reutilizar de forma incorrecta.
- Los pasos “Given” se utilizan para establecer un estado inicial, no para denotar un comportamiento.

X

Bad example! Do not copy

X

=

- **Feature:** Google Searching
- **Scenario:** Google search result page elements
- **Given** the user navigates to the Google home page
- **When** the user entered “panda” at the search bar
- **Then** the results page shows links related to “panda”
- **And** image links for “panda”
- **And** video links for “panda”
- **Feature:** Google Searching
- **Scenario:** Simple Google search
- **Given** the user navigates to the Google home page
- **When** the user entered “panda” at the search bar
- **Then** links related to “panda” will be shown on the results page



✓

The right way

✓

≡

- **Feature:** Google Searching
- **Scenario:** Simple Google search
- **Given** a web browser is at the Google home page
- **When** the user enters “panda” into the search bar
- **Then** links related to “panda” are shown on the results page



“Given” should always use present or present perfect tense, and “When” and “Then” should always use present tense.

Q



Avoid assertion language

- **Evita palabras “verify”, “assert” o “should”** en títulos de escenarios. Esas palabras ponen mas énfasis en la aseveración más que en el comportamiento, además se torna un título repetitivo.

✗

Bad example! Do not copy

Verify the user can change their address on the profile page.

Assert that a stock quote is displayed in green text when its value is higher than its previous closing value.

The goodbye page should be displayed after a successful logout.

✓

The right way

Profile page address change.

A stock quote has green text when its value is higher than its previous closing value.

Logout displays the goodbye page.

